

Experiment 10

Aim:- Write a python script to implement Naïve Bayes Classifier.

Code:-

```
from sklearn import datasets
from sklearn import metrics
from sklearn.naive_bayes import GaussianNB

dataset=datasets.load_iris()
model=GaussianNB()
model.fit(dataset.data, dataset.target)

print(model)
GaussianNB(priors=None)

expected=dataset.target
predicted=model.predict(dataset.data)

print(metrics.classification_report(expected, predicted))
print(metrics.confusion_matrix(expected, predicted))
```

Output:-

```
... GaussianNB()
      precision    recall  f1-score   support

      0         1.00      1.00      1.00        50
      1         0.94      0.94      0.94        50
      2         0.94      0.94      0.94        50

 accuracy          0.96          150
 macro avg         0.96          150
 weighted avg      0.96          150

[[50  0  0]
 [ 0 47  3]
 [ 0  3 47]]
```

Experiment 11

Aim:- Write a python script to implement Beam Search Algorithm.

Code:-

```
import queue

def beam_search(graph, start, goal, beam_width):
    open_set, closed_set, search_tree = queue.PriorityQueue(), set(), {start: None}
    open_set.put((0, start))

    while not open_set.empty():
        current_cost, current_node = open_set.get()
        if current_node == goal: # Goal condition
            path = []
            while current_node:
                path.insert(0, current_node)
                current_node = search_tree[current_node]
            return path
        closed_set.add(current_node)

        for successor, cost in sorted(graph.get(current_node, []), key=lambda x:
x[1][:beam_width]):
            if successor not in closed_set:
                open_set.put((current_cost + cost, successor))
                search_tree[successor] = current_node

    return None

# Example graph and parameters
graph = {
    'A': [('E', 4), ('C', 2), ('I', 4)],
```

```
'C': [('F', 1), ('G', 2), ('H', 8)],  
'E': [('F', 2), ('C', 2), ('H', 5)],  
'F': [('G', 3), ('H', 4)],  
'G': [('H', 1)],  
'T': [('E', 6), ('H', 7)],  
}  
start_node, goal_node, beam_width = 'A', 'H', 2  
path = beam_search(graph, start_node, goal_node, beam_width)  
print("Shortest Path:", ' -> '.join(path) if path else "No path found.")
```

Output:-

```
Shortest Path: A -> C -> F -> G -> H
```

Experiment 9

Aim:- Write a python script to implement Tic Tac Toe Game.

Code:-

```
# Tic Tac Toe Game

def print_board(board):
    for row in board:
        print(" | ".join(row))
    print("-" * 9)

def check_winner(board):
    # Check rows, columns, and diagonals for a win
    for i in range(3):
        if board[i][0] == board[i][1] == board[i][2] != " " or \
            board[0][i] == board[1][i] == board[2][i] != " ":
            return True
    return board[0][0] == board[1][1] == board[2][2] != " " or \
        board[0][2] == board[1][1] == board[2][0] != " "

def tic_tac_toe():
    board = [ [" " for _ in range(3)] for _ in range(3) ]
    current_player = "X"

    for turn in range(9):
        print_board(board)
        print(f"Player {current_player}'s turn.")
        row, col = map(int, input("Enter row and column (0, 1, 2): ").split())
```

```

if board[row][col] == " ":
    board[row][col] = current_player
    if check_winner(board):
        print_board(board)
        print(f"Player {current_player} wins!")
        return
    current_player = "O" if current_player == "X" else "X"
else:
    print("Cell already taken. Try again!")

print_board(board)
print("It's a draw!")

# Start the game
tic_tac_toe()

```

Output:-

```

  |  |
  |  |
  |  |
-----
Player X's turn.
X |  |
  |  |
  |  |
-----
Player O's turn.
X |  |
  | O |
  |  |
-----
Player X's turn.
Cell already taken. Try again!
X |  |
  | O |
  |  |
-----

```

```

Player X's turn.
X | X |
  | O | X
  |  | O
-----
Player O's turn.
Cell already taken. Try again!
X | X |
  | O | X
  |  | O
-----
Player O's turn.
X | X |
O | O | X
  |  | O
-----
Player X's turn.
X | X | X
O | O | X
  |  | O
-----
Player X wins!

```

Experiment 8

Aim:- Implement the tokenization, remove stopwords, stemming and bag of words using nltk.

Code:-

```
import nltk

from nltk.tokenize import word_tokenize

from nltk.corpus import stopwords

from nltk.stem import PorterStemmer

from sklearn.feature_extraction.text import CountVectorizer

nltk.download('punkt', quiet=True)

nltk.download('stopwords', quiet=True)

text = "Running is a great way to stay fit, and it is also fun!"

tokens = word_tokenize(text)

filtered = [w for w in tokens if w.lower() not in
set(stopwords.words('english'))]

stemmed = [PorterStemmer().stem(w) for w in filtered]

boW = CountVectorizer().fit_transform([' '.join(stemmed)]).toarray()

print("Tokens:", tokens)

print("Filtered:", filtered)

print("Stemmed:", stemmed)

print("Bag of Words:\n", boW)
```

Output:-

```
Tokens: ['Running', 'is', 'a', 'great', 'way', 'to', 'stay', 'fit', ',', 'and', 'it', 'is', 'also', 'fun', '!']
Filtered: ['Running', 'great', 'way', 'stay', 'fit', ',', 'also', 'fun', '!']
Stemmed: ['run', 'great', 'way', 'stay', 'fit', ',', 'also', 'fun', '!']
Bag of Words:
[[1 1 1 1 1 1 1]]
```

Experiment 7

Aim:- To program best fit distributions for at least 2 discrete dataset and estimate mean and variance for the dataset.

Code:-

```
import numpy as np
from scipy.stats import poisson, binom

# Sample data
data = np.random.randint(0, 10, 100)

# Poisson fit (mean is  $\lambda$ )
 $\lambda$  = np.mean(data)
mean_poisson, var_poisson =  $\lambda$ ,  $\lambda$  # Poisson mean and variance are both  $\lambda$ 

# Binomial fit (n=max, p= $\lambda$ /n)
n = max(data)
p =  $\lambda$  / n
mean_binom, var_binom = n * p, n * p * (1 - p)

print(f'Poisson - Mean: {mean_poisson}, Variance: {var_poisson}')
print(f'Binomial - Mean: {mean_binom}, Variance: {var_binom}')
```

Output:-

```
Poisson - Mean: 4.57, Variance: 4.57
Binomial - Mean: 4.57, Variance: 2.2494555555555555
```

Experiment 6

Aim:- To program best fit distributions for at least 2 continuous and estimate mean and variance for the dataset.

Code:-

```
import numpy as np
from scipy.stats import norm, expon

# Sample data (continuous)
data = np.random.normal(loc=5, scale=2, size=100)

# Normal fit (mean and variance estimated from the data)
mean_norm, var_norm = np.mean(data), np.var(data)

# Exponential fit (mean = 1/lambda)
lambda_exp = 1 / np.mean(data)
mean_exp, var_exp = 1 / lambda_exp, 1 / lambda_exp**2

# Print results
print(f'Normal - Mean: {mean_norm}, Variance: {var_norm}')
print(f'Exponential - Mean: {mean_exp}, Variance: {var_exp}')
```

Output:-

```
Normal - Mean: 4.505624019661631, Variance: 3.754074650965322
Exponential - Mean: 4.505624019661631, Variance: 20.300647806551833
```


Experiment 5

Aim:- To extract intra-day stock market data for any 4 stocks of your choice. Write a python script that can fetch the data, plot the values, derive mean and correlation. Also conduct linear regression for the same.

Code:-

```
import yfinance as yf
import matplotlib.pyplot as plt
import numpy as np
from scipy import stats

# Extract intra-day stock market data for 4 stocks
stock_1 = ['AAPL', 'MSFT', 'GOOGL', 'AMZN']
# Download the data
data = yf.download(stock_1, period='5d', interval='1m')
yf.download(stock_1, period='5d', interval='1m')
close_prices = data['Close']
close_prices
plt.figure(figsize=(14, 7))
for stock_2 in stock_1:
    plt.plot(close_prices.index, close_prices[stock_2], label=stock_2)
plt.xlabel('Time')
plt.ylabel('Price')
# These functions label the x-axis (Time) and y-axis (Price) of the plot.
plt.title('Intra-day Stock Prices')
plt.legend()
plt.show()
plt.plot(close_prices.index, close_prices[stock_2], label=stock_2)
stock_1s = ['AAPL', 'MSFT', 'GOOGL', 'AMZN'] # Example stock symbols
```

```

# Download the data
data = yf.download(stock_1s, period='5d', interval='1m')
# Extract the 'Close' price for simplicity
close_prices = data['Close']
# Step 2: Plot the values for the 4 stocks
plt.figure(figsize=(14, 7))
for stock_1 in stock_1s:
    plt.plot(close_prices.index, close_prices[stock_1], label=stock_1)
    plt.xlabel('Time')
    plt.ylabel('Price')
    plt.title('Intra-day Stock Prices')
    plt.legend()
    plt.show()
# Step 3: Develop linear regression, derive mean, and correlation
# Calculate mean and correlation for each stock
mean_prices = close_prices.mean()
correlation_matrix = close_prices.corr()
print("Mean Prices:")
print(mean_prices)
print("\nCorrelation Matrix:")
print(correlation_matrix)
# Perform linear regression for a sample pair of stocks
# We'll take AAPL as x and MSFT as y for linear regression
slope, intercept, r_value, p_value, std_err =
stats.linregress(close_prices['AAPL'],
close_prices['MSFT'])
print(f"\nLinear Regression between AAPL and MSFT:")
print(f"Slope: {slope}")
print(f"Intercept: {intercept}")

```

```

print(f'R-squared: {r_value**2}')

# Plot the regression line along with the data points
plt.figure(figsize=(10, 6))

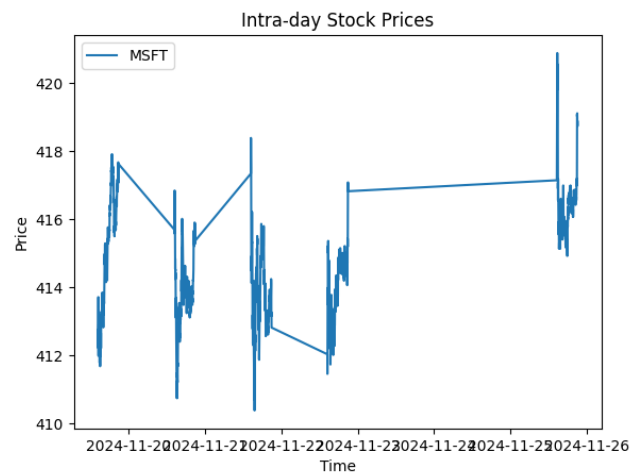
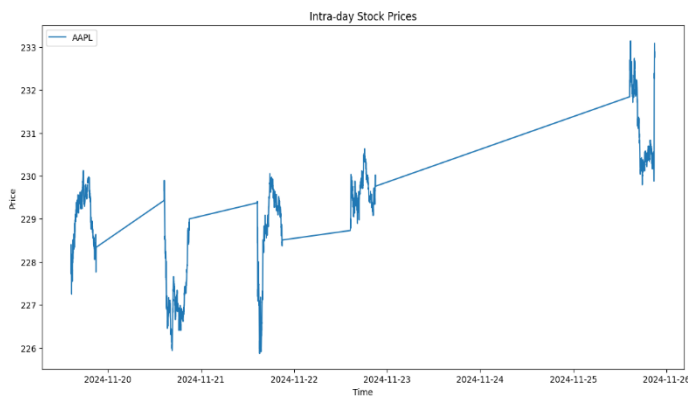
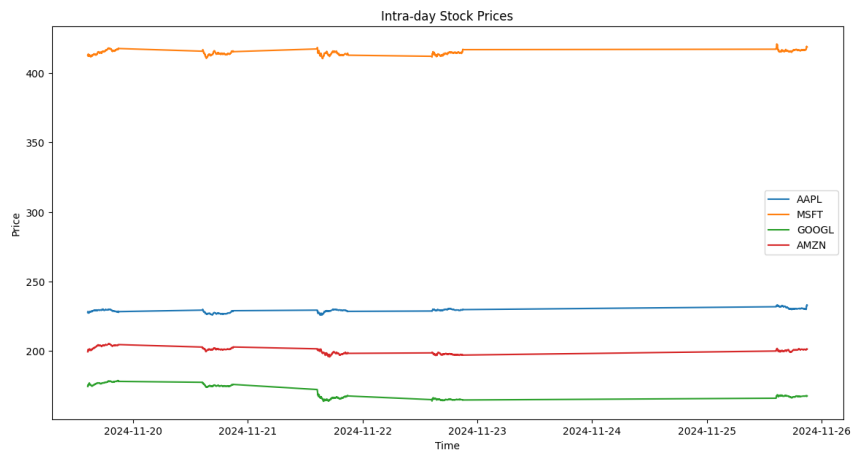
plt.scatter(close_prices['AAPL'], close_prices['MSFT'], label='Data Points')

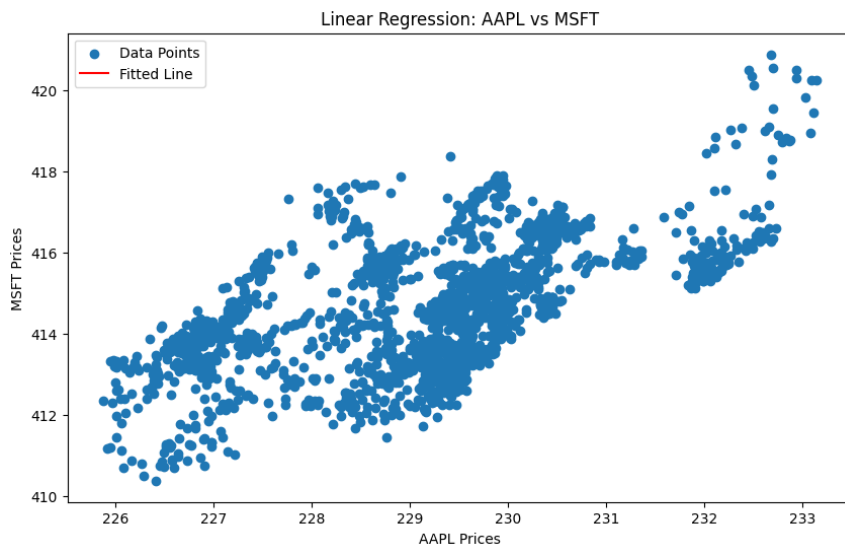
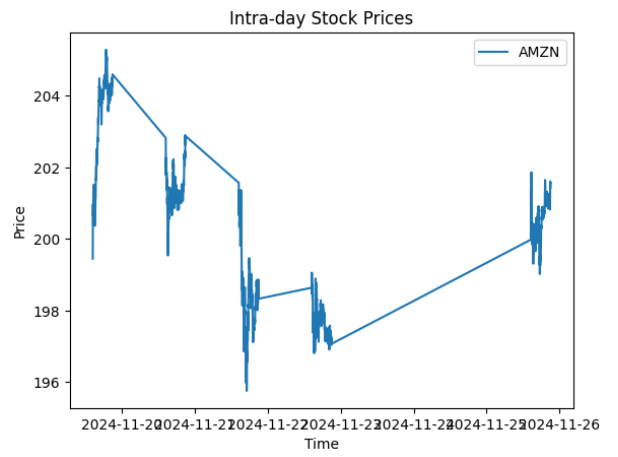
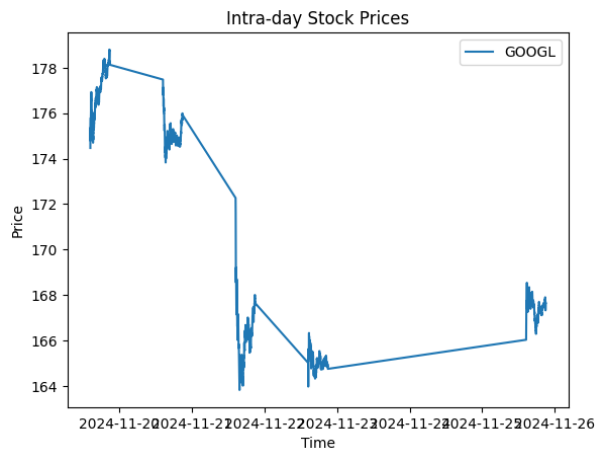
plt.plot(close_prices['AAPL'], intercept + slope * close_prices['AAPL'], 'r',
label='Fitted Line')

plt.xlabel('AAPL Prices')
plt.ylabel('MSFT Prices')
plt.title('Linear Regression: AAPL vs MSFT')
plt.legend()
plt.show()

```

Output:-





Experiment 2

Aim :- To implement the A* algorithm code.

Code:-

```
import heapq

def a_star(grid, start, goal):
    def heuristic(a, b): # Manhattan distance
        return abs(a[0] - b[0]) + abs(a[1] - b[1])

    open_set = [(0, start)] # Priority queue
    came_from = {} # To reconstruct the path
    cost = {start: 0} # G-score

    while open_set:
        _, current = heapq.heappop(open_set)
        if current == goal:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            return path[::-1] + [goal]

        for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]: # Directions
            neighbor = (current[0] + dx, current[1] + dy)
            if 0 <= neighbor[0] < len(grid) and 0 <= neighbor[1] < len(grid[0])
            and grid[neighbor[0]][neighbor[1]] == 0:
```

```

        new_cost = cost[current] + 1
        if neighbor not in cost or new_cost < cost[neighbor]:
            cost[neighbor] = new_cost
            priority = new_cost + heuristic(neighbor, goal)
            heapq.heappush(open_set, (priority, neighbor))
            came_from[neighbor] = current

    return None # No path found

# Example usage
grid = [
    [0, 1, 0, 0],
    [0, 1, 0, 1],
    [0, 0, 0, 1],
    [1, 1, 0, 0]
]
start, goal = (0, 0), (3, 3)
print(a_star(grid, start, goal))

```

Result:-

```
[(1, 0), (2, 0), (2, 1), (2, 2), (3, 2), (3, 3), (3, 3)]
```

Experiment 4

Aim:-To implement sentiment analysis using the Linear Regression Model of sklearn.

Code:-

```
import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Example dataset (replace with your dataset)
data = pd.DataFrame({
    'text': ['I love programming', 'This is terrible', 'I enjoy learning', 'I hate bugs'],
    'sentiment': [1, -1, 1, -1] # 1 for positive, -1 for negative
})

# Feature extraction
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(data['text'])
y = data['sentiment']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Train Linear Regression model
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
# Predict and evaluate
```

```
y_pred = model.predict(X_test)
```

```
print(y_pred)
```

Output:-

```
[0.33333333]
```

```
MSE: 1.7777777777777777
```

```
Predictions: [0.66666667 0.33333333]
```


Experiment 3

Aim:-To use lobe.ai software and train it using your custom dataset.

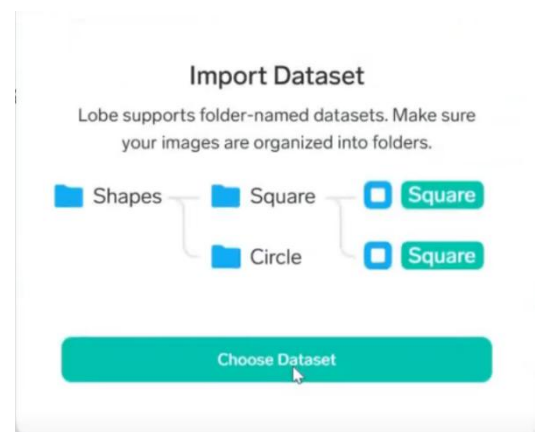
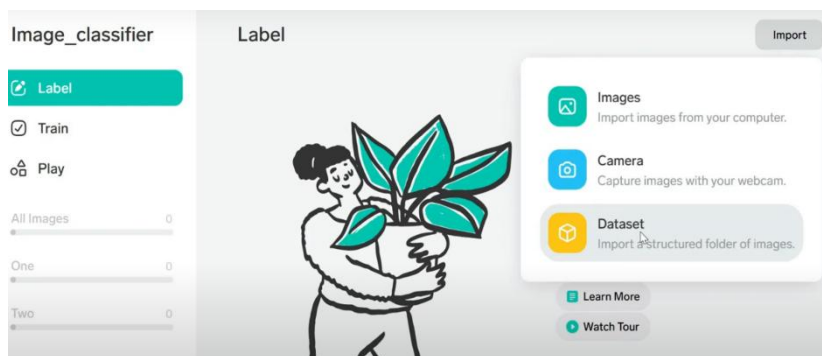
Implementation:-

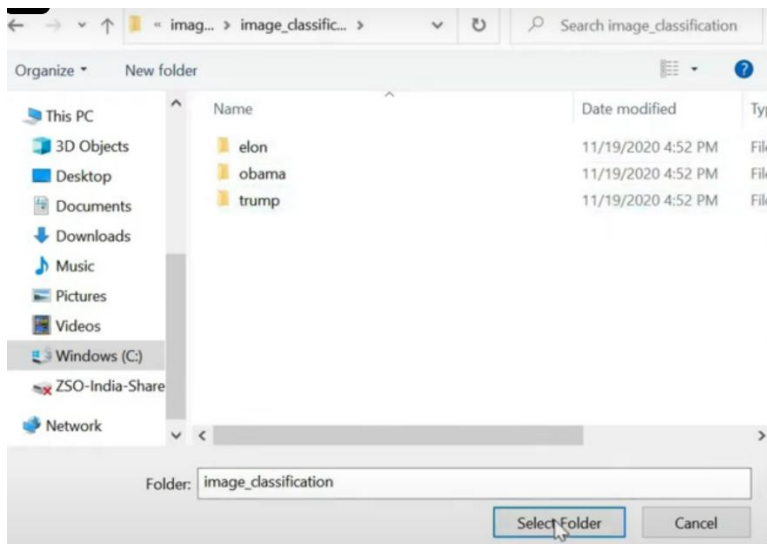
Steps1) Download and Install Lobe.ai

- Download the Lobe software from [Lobe.ai](https://lobe.ai).
- Install it on your computer (Windows or macOS).

Step 2) Import Your Dataset

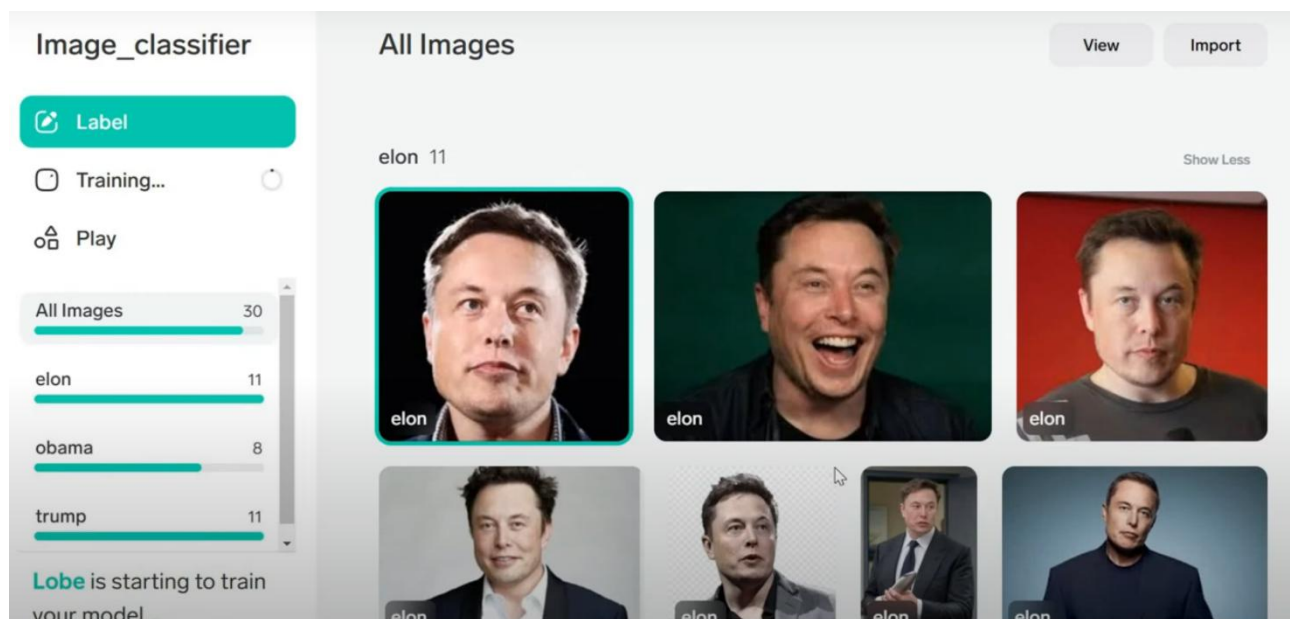
- Open Lobe.
- Click Start a New Project.
- Drag and drop your dataset folder into Lobe, or select it manually.
- Lobe automatically detects the labels based on folder names.





Step 3) Train Your Model

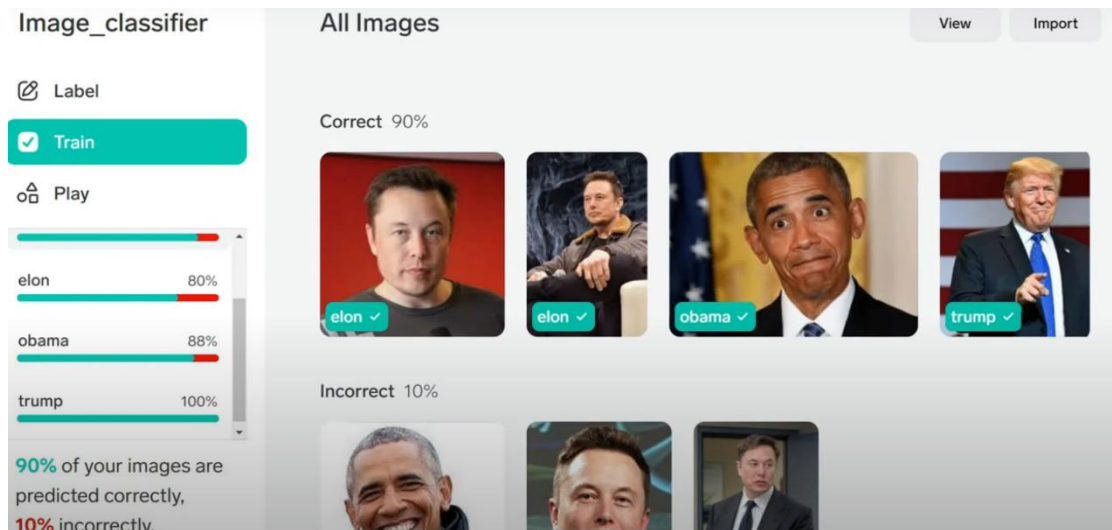
- Lobe starts training as soon as you import data.
- You can monitor training progress in real-time.
- Lobe will automatically adjust settings to optimize the model.



Step 4) Test Your Model

- Use the built-in testing interface to upload or capture new images and see how well the model performs.

- Lobe provides visual feedback for predictions and confidence scores.



Result:-We successfully learned to train the model and do image classification using Lobe.ai tool.